Distributed Systems
Java RMI (*Remote Method Invocation*)

Alberto Lafuente

Mikel Larrea

UPV/EHU

Contents

- Interface
- Implementation
- Server
- Client
- Application deployment:
  - Java compiler
  - RMI compiler
  - Launching the name service of RMI
  - Launching the server
  - Launching the client

Interface

- Definition of the objects that will be accesses remotely:
  - methods, with their parameters
- The implementation of the methods is not given
- Necessary to extend the interface java.rmi.Remote
- In all the remote methods, the possibility of throwing the exception java.rmi.RemoteException must be indicated

Interface (Hello.java)

```
package hello;

import java.rmi.Remote;
import java.rmi.RemoteException;
import java.util.Date;

public interface Hello extends Remote {
   String sayHello() throws RemoteException;
   Date getDate() throws RemoteException;
}
```

Implementation

- It is a normal Java class
- Extends the class java.rmi.server.UnicastRemoteObject, and implements the interface defined in the previous step
  - Implements all the remote methods of the interface
- In the constructor method of this class only the constructor of the class UnicastRemoteObject is called: *super()*

Implementation (HelloImpl.java)

```
package hello;

import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.RMISecurityManager;
import java.rmi.server.UnicastRemoteObject;
import java.util.Date;

public class HelloImpl extends UnicastRemoteObject
   implements Hello {

   public HelloImpl() throws RemoteException {
      super();
   }
   public String sayHello() {
      return "Kaixo Mikel!";
   }
   public Date getDate() {
      return new Date();
   }
   // ...
```

Server

- Starts a security manager
- Creates the object that will be accessed remotely
  - It is an instance of the implementation class
- Registers the remote object (with a given name) in the name service of RMI, by means of the method Naming.rebind
- The server can be implemented in a separate class, or also within the implementation class (*main* method)

Server (HelloImpl.java)

```
   // ...

   public static void main(String args[]) {

      // Create and install a security manager
      System.setSecurityManager(new RMISecurityManager());

      try {
         Hello obj = new HelloImpl();
         // Bind this object instance to the name "HelloServer"
         Naming.rebind("HelloServer", obj);
         System.out.println("HelloServer bound in registry");
      } catch (Exception e) {
         System.out.println("HelloImpl exception: " +
                            e.getMessage());
         e.printStackTrace();
      }
   }
}
```

Client

- Starts a security manager
- Gets a reference to the remote object that has to be accessed, by means of a request to the name service of RMI
  - Method Naming.lookup, using the same name with which the remote object was registered
- Once the reference is obtained, it invokes the methods of the remote object as if it were a local object, receiving the results and showing then on the screen

Client (HelloClient.java)

```
package hello;

import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.RMISecurityManager;
import java.util.Date;

public class HelloClient {

   public static void main(String args[]) {

      if (args.length != 1) {
         System.out.println(("Usage: java HelloClient host");
         Systm.exit(1);
      }

      System.setSecurityManager(new RMISecurityManager());
      // ...
```

Client (HelloClient.java)

```
      // ...
      try {
         Hello obj = (Hello)Naming.lookup("rmi://" + args[0] +
                                          "/HelloServer");
         String str = obj.sayHello();
         System.out.println(str);
         // invoke method on server object and locally
         // (to compare dates)
         Date d_remote = obj.getDate();
         Date d_local = new Date();
         System.out.println("Date on server is " + d_remote);
         System.out.println("Date on client is " + d_local);
      } catch (Exception e) {
         System.out.println("HelloClient exception: " +
                            e.getMessage());
         e.printStackTrace();
      }
   }
}
```

Application deployment

- Java compiler:
  - javac hello/*.java
- RMI compiler (on the implementation class):
  - rmic hello.HelloImpl
- Launching the name service of RMI (included in the Java distribution):
  - rmiregistry
- Launching the server:
  - java –Djava.security.policy=hello/policy hello.HelloImpl
- Launching the client:
  - java –Djava.security.policy=hello/policy hello.HelloClient host

Exercise 1

- Develop a Java RMI power calculation service. The operations offered by the service will be the following:
  - Square calculation operation:

```
long square(int n);
```

  - Power calculation operation:

```
long power(int n1, int n2);
```

Exercise 2

- Test between two machines of the laboratory the RMI example that simulates Cristian's clock synchronization algorithm
  - Filename: `cristian.tar`
- Adapt the previous example to simulate the NTP symmetric algorithm
  - Hint: two time values of each machine must be obtained